Tool Use y MCP en ADK

Material técnico de apoyo — Módulo 4

FunctionTool · AgentTool · MCPToolset · FastMCP · MCP Toolbox

Agenda

0. Intro: qué es MCP
1. FunctionTool: el tool nativo
2. AgentTool: un agente como tool
3. MCPToolset / McpToolset: el cliente MCP de ADK
4. FastMCP: escribir un servidor MCP propio
5. MCP Toolbox for Databases: SQL como servidor MCP, sin código
6. MCP en la práctica: dos ejemplos reales (software-bug-assistant, travel-concierge)

0. ¿Qué es MCP?

Model Context Protocol: estándar abierto para que los LLMs (Gemini, Claude, ...) se comuniquen con apps externas, fuentes de datos y tools — un mecanismo de conexión universal, no un adaptador a medida por integración.

Arquitectura **cliente-servidor**:

- **Servidor MCP** expone *resources* (datos), *prompts* (plantillas) y *tools* (funciones ejecutables).
- **Cliente MCP** las consume — puede ser una app host de LLM, o un agente.

Dos patrones de integración con ADK:

1. **ADK como cliente MCP** — consumir servidores existentes (todo lo que vimos este módulo).
2. **ADK como servidor MCP** — envolver tools de ADK para que cualquier cliente

MCP — protocolo vs. librería

- **MCP** es una especificación de protocolo: reglas de comunicación (JSON-RPC, `initialize` / `notifications/initialized`, `tools/list`, `tools/call`).
- **ADK** es una librería Python para construir agentes.
- `McpToolset` es el puente: implementa el **lado cliente** del protocolo dentro de ADK.

Para escribir un servidor MCP en Python: paquete **`mcp`** (`pip install mcp` — SDK oficial de Anthropic, no `model-context-protocol`).

MCP — naturaleza asíncrona (con matiz)

- El **servidor MCP** y el interior de `McpToolset` son asíncronos de punta a punta (`async def` en los handlers).
- **Los tools de ADK del lado cliente NO necesitan ser `async`:** `FunctionTool` soporta funciones sync o async por igual. Todos los tools de este módulo (`get_order_status`, `get_current_weather`, ...) son funciones sync normales.

No hace falta escribir `async def` en tus tools solo porque MCP esté en la cadena.

MCP — sesiones con estado

MCP establece conexiones **persistentes y con estado**, muy distinto de una API REST típica sin estado. El handshake `initialize` → `notifications/initialized` tiene que completarse antes de aceptar cualquier otro request.

Implicancia de despliegue: complica escalar servidores remotos para muchos usuarios — el diseño original asumía cliente y servidor coubicados. Requiere decisiones de infra (balanceo, session affinity) que una REST API stateless no necesita.

MCP — quién gestiona el ciclo de vida de la conexión

En ADK 2.2.0, el manejo de la conexión (`AsyncExitStack`) es **interno** a `McpToolset` — no algo que escribas a mano. Lo que sí es tu responsabilidad: cerrar el `Runner` .

```
await runner.close()   # cierra toolsets MCP, plugins, flushea la sesión
# equivalente:
async with Runner(agent=agent, ...) as runner:
    ...
```

Persistencia de sesión: `McpToolset` implementa `__getstate__` / `__setstate__` para serializarse (útil en Cloud Run/GKE) — pero las conexiones MCP activas **no se restablecen automáticamente** al restaurar: se re-inicializan cuando hacen falta.

1. FunctionTool — qué es

- Envuelve una función Python nativa para que un agente la use como tool.
- ADK genera el schema (`FunctionDeclaration`) automáticamente a partir de:
  - **type hints** de los parámetros
  - **docstring** de la función
- `tools=[mi_funcion]` y `tools=[FunctionTool(func=mi_funcion)]` son lo mismo para una función simple.

FunctionTool — ¿wrapping implícito o explícito?

Verificado en vivo: `tools=[func]` y `tools=[FunctionTool(func=func)]` son **exactamente lo mismo** en runtime — `agent.canonical_tools()` envuelve la función pelada igual, y los callbacks (`before_tool_callback`, `after_tool_callback`, `on_tool_error_callback`) viven en el `Agent`, no en el tool: se disparan idéntico en los dos casos.

`ToolContext` también funciona igual en ambos casos — depende de la firma de la función, no del wrapper.

Solo hay dos razones reales para escribir un tool explícito: `require_confirmation` y `LongRunningFunctionTool` — ninguna alcanzable con la función pelada.

FunctionTool — ejemplo: ToolContext

```python
from google.adk.agents import Agent
from google.adk.tools import FunctionTool, ToolContext

def query_vector_store(tool_context: ToolContext, query: str, top_k: int = 5) -> dict:
    """Consulta la base vectorial del tenant actual."""
    tenant_id = tool_context.state.get("tenant_id")   # nunca lo decide el LLM
    if not tenant_id:
        return {"error": "Falta tenant_id en el estado."}
    top_k = min(top_k, 20)                                  # límite de seguridad
    return {"tenant_id": tenant_id, "results": [f"doc_{i}" for i in range(top_k)]}

agent = Agent(
    name="retrieval_agent", model="gemini-2.5-flash",
    tools=[query_vector_store],   # tools=[FunctionTool(func=query_vector_store)] sería idéntico
)
```

FunctionTool — ejemplo: guardrail + error estructurado

```
def guardrail_before_tool(tool, args, tool_context):
    """Devolver un dict corta la ejecución real; None deja pasar."""
    if "DROP TABLE" in str(args.get("payload", "")).upper():
        return {"success": False, "error_type": "BLOCKED_BY_GUARDRAIL"}
    return None

def error_fallback(tool, args, tool_context, error):
    """Convierte una excepción cruda en algo que el LLM puede razonar."""
    if isinstance(error, TimeoutError):
        return {"success": False, "error_type": "TIMEOUT_ERROR", "retryable": True}
    return {"success": False, "error_type": "INTERNAL_FAILURE"}

agent = Agent(
    name="resilient_agent", model="gemini-2.5-flash",
    tools=[execute_system_query],  # función pelada — los callbacks igual se disparan
    before_tool_callback=guardrail_before_tool,
    on_tool_error_callback=error_fallback,
)
```

FunctionTool — razón real #1: require_confirmation

No tiene equivalente si pasás la función pelada. Activa confirmación humana antes de ejecutar el tool — útil para acciones irreversibles (borrar, pagar, enviar).

```
def cancel_subscription(subscription_id: str) -> dict:
    """Cancela una suscripción de forma irreversible."""
    ...

cancel_tool = FunctionTool(func=cancel_subscription, require_confirmation=True)
```

FunctionTool — razón real #2: LongRunningFunctionTool

Para operaciones que no terminan en la misma llamada (aprobación humana, proceso batch). Es una **subclase** de `FunctionTool` — no un parámetro, hace falta la clase.

```python
from google.adk.tools import LongRunningFunctionTool, ToolContext

def get_user_choice(options: list[str], tool_context: ToolContext) -> str | None:
    """Le muestra las opciones al usuario y pide que elija una."""
    tool_context.actions.skip_summarization = True
    return None   # nada que resolver todavía — la respuesta llega después

get_user_choice_tool = LongRunningFunctionTool(func=get_user_choice)
```

(Ejemplo real, tomado del propio código fuente de ADK — `get_user_choice_tool.py`.)

LongRunningFunctionTool — qué hace por debajo

1. Marca `is_long_running = True` en el tool.
2. Le agrega a la descripción una nota automática: *"No vuelvas a llamar este tool si ya devolvió un estado pendiente."*
3. La función devuelve un resultado **inicial** (o `None`) — el agente se pausa ahí sin bloquear el proceso.

El resultado final llega **de afuera** (un humano, otro sistema), no de una segunda ejecución de la función.

LongRunningFunctionTool — cómo se resume (patrón real de `adk run`)

```
# 1. Detectar la pausa: un evento con long_running_tool_ids
async for event in runner.run_async(..., new_message=content):
    if event.long_running_tool_ids:
        for part in event.content.parts:
            fc = part.function_call
            if fc and fc.id in event.long_running_tool_ids:
                pending_id, pending_name = fc.id, fc.name

# 2. Resumir: mandar la respuesta real apuntando al mismo function_call_id
resume = types.Content(role="user", parts=[types.Part(
    function_response=types.FunctionResponse(
        id=pending_id, name=pending_name, response={"status": "approved"}))])

async for event in runner.run_async(..., new_message=resume):
    ...   # el agente retoma como si el tool hubiese devuelto esto directamente
```

No se re-ejecuta la función — ADK inyecta la respuesta por `id`.

FunctionTool — envolver una API externa

Dos estrategias:

1. **Wrapper de función Python** (`httpx` / `requests` adentro de la función) — la más común.
2. **`OpenAPIToolset`**: le pasás una spec OpenAPI y ADK descubre todos los endpoints como tools, sin escribir una función por endpoint.

```
from google.adk.tools.openapi_tool import OpenAPIToolset

api_toolset = OpenAPIToolset(spec_str=spec_str, spec_str_type="json")
agent = Agent(model="gemini-2.5-flash", tools=[api_toolset])
```

FunctionTool — auth sin exponer el secreto al modelo

- **Wrapper Python:** leer el token de `tool_context.state` dentro de la función.
- **`OpenAPIToolset`:** parámetro `header_provider` — un callback que corre en cada request HTTP e inyecta headers dinámicamente por sesión/usuario.

```
def inject_bearer_token(readonly_context) -> dict:
    token = readonly_context.state.get("oauth_access_token")
    return {"Authorization": f"Bearer {token}"} if token else {}

api_toolset = OpenAPIToolset(spec_str=spec_str, header_provider=inject_bearer_token)
```

El modelo nunca ve el token — ni en el prompt, ni en el schema.

2. AgentTool — un agente como tool

- Envuelve un **agente ADK completo** y lo expone como tool de otro agente "padre".
- El padre ve un tool con una entrada y una salida — no ve sub-conversaciones.
- Distinto de `sub_agents=[...]`: con `AgentTool` el padre **llama** (como función); con `sub_agents` **transfiere el control** del turno completo.

```
AgentTool(agent: BaseAgent, skip_summarization: bool = False, *,
          include_plugins: bool = True, propagate_grounding_metadata: bool = False)
```

AgentTool — cómo arma el schema

- Nombre y descripción del tool salen de `agent.name` / `agent.description`.
- Por default expone una sola entrada de texto libre: `{"request": "string"}`.
- Si el sub-agente define `input_schema` (Pydantic), esos campos reemplazan al `request` genérico.

`skip_summarization=True`: el resultado del sub-agente pasa **intacto** al padre, sin que su LLM lo reformule. Clave cuando el sub-agente devuelve un `output_schema` estructurado (ej. un itinerario en JSON).

AgentTool — ejemplo: Google Search aislado

```python
from google.adk.agents import Agent
from google.adk.tools import google_search
from google.adk.tools.agent_tool import AgentTool

search_agent = Agent(
    model="gemini-2.5-flash", name="search_agent",
    instruction="Sos un especialista en Google Search.",
    tools=[google_search],
)
search_tool = AgentTool(search_agent)

support_agent = Agent(
    model="gemini-2.5-flash", name="support_agent",
    instruction="Usá search_agent si hace falta info de internet.",
    tools=[search_tool],
)
```

`google_search` solo puede usarse en un agente de un único tool — por eso se

AgentTool vs. FunctionTool vs. MCPToolset

	FunctionTool	AgentTool	MCPToolset
Envuelve	una función Python	un agente ADK completo	un servidor MCP externo
Schema sale de	type hints + docstring	`agent.name` / `description` o `input_schema`	el servidor MCP (`tools/list`)
Ejecuta	la función, mismo proceso	un `Runner` completo sobre el sub-agente	un `tools/call` contra proceso externo

Mismo lugar en `tools=[...]`, origen completamente distinto.

3. MCPToolset / McpToolset — el cliente MCP de ADK

- Dado un servidor MCP (propio o de terceros, en cualquier lenguaje), se conecta, descubre sus tools, y los expone al agente igual que un `FunctionTool` nativo.
- **`MCPToolset`** (mayúsculas) está **deprecado** en `google-adk` 2.2.0 → usar **`McpToolset`** (minúsculas). Mismos parámetros, solo cambió el nombre.

```
McpToolset(*, connection_params, tool_filter=None, tool_name_prefix=None,
           auth_scheme=None, auth_credential=None, header_provider=None,
           require_confirmation=False, use_mcp_resources=False)
```

MCPToolset — los tres transportes

Clase	Transporte	Ejemplo real
`StdioServerParameters` / `StdioConnectionParams`	subproceso local (STDIO)	`weather_agent`, Google Maps MCP, Airbnb MCP
`SseConnectionParams`	HTTP + Server-Sent Events	—
`StreamableHTTPConnectionParams`	HTTP streaming (recomendado)	GitHub MCP remoto (`software-bug-assistant`)

```
StdioConnectionParams(
    server_params=StdioServerParameters(command="python3", args=["weather_mcp_server.py"]),
    timeout=15,
```

MCPToolset — tool_filter y header_provider

`tool_filter`: allowlist de nombres — el servidor puede exponer más de lo que tu agente necesita.

```
McpToolset(
    connection_params=StreamableHTTPConnectionParams(url="https://api.githubcopilot.com/mcp/"),
    tool_filter=["search_issues", "get_issue"],   # nada de escritura
)
```

`header_provider`: headers dinámicos por sesión, sin fijar un token estático en la construcción — mismo mecanismo que vimos en `OpenAPIToolset`.

MCPToolset — patrón defensivo (try/except)

Si el servidor no está disponible (falta `npx`, proceso caído, falta un token), el agente **igual carga**, solo sin ese tool:

```
try:
    weather_tools = McpToolset(connection_params=StdioConnectionParams(...))
except Exception:
    weather_tools = None

tools = [t for t in [weather_tools] if t is not None]
root_agent = Agent(model="gemini-2.5-flash", tools=tools)
```

Mismo principio de resiliencia del Ejercicio 4 del lab, aplicado a *disponibilidad del tool*.

MCPToolset — qué hace ADK por debajo

1. **Descubrimiento** — `initialize` + `tools/list` contra el servidor.
2. **Traducción de schema** — `inputSchema` (JSON Schema) → `FunctionDeclaration`, indistinguible de un tool nativo para el LLM.
3. **Adaptación de la llamada** — la decisión del modelo se traduce a `tools/call`; la respuesta MCP vuelve como el `return` de una función Python.

Para el LLM y el código del agente: un tool nativo, uno de LangChain, y uno MCP son **la misma entrada** en `tools=[...]`.

4. FastMCP — el framework para escribir servidores MCP

- **No es de Google ni de ADK** — framework de terceros para el SDK oficial `mcp` de Anthropic.
- Inspirado en la ergonomía de FastAPI: decoradores + type hints + docstrings → schema automático.
- Solo aparece del lado **servidor**, y solo si vos escribís ese servidor.

```
from fastmcp import FastMCP
mcp = FastMCP("weather-mcp-server")

@mcp.tool
def get_current_weather(city: str) -> dict:
    """Get the current weather for a city."""
    ...
```

FastMCP — validación por tipos (Literal → enum)

```python
from typing import Literal

City = Literal["Buenos Aires", "Córdoba", "Rosario", "Mendoza", "Salta"]

@mcp.tool
def get_current_weather(city: City, unit: Literal["celsius", "fahrenheit"] = "celsius") -> dict:
    """Get the current weather conditions for an Argentine city."""
    ...
```

`Literal[...]` se traduce a `enum` en el `inputSchema`. Si el modelo manda una ciudad inválida, **Pydantic la rechaza antes** de ejecutar la función — el rechazo es de schema, no un `{"error": ...}` armado a mano.

FastMCP — las tres primitivas MCP

Decorador	Primitiva	Para qué
`@mcp.tool`	Tool	acción/cómputo invocable por el LLM
`@mcp.resource("uri://...")`	Resource	dato de contexto direccionado por URI
`@mcp.prompt`	Prompt	plantilla de mensaje parametrizada

`lab.md` solo usa `@mcp.tool` porque en un agente conversacional es la primitiva que importa.

FastMCP — convertir una API existente

```python
from fastapi import FastAPI
from fastmcp import FastMCP

app = FastAPI(title="Enterprise Metrics API")

@app.get("/status")
def get_status() -> dict:
    """Estado de salud del servicio."""
    return {"status": "ok"}

mcp = FastMCP.from_fastapi(app=app, name="metrics-server")
mcp.run()
```

Cada ruta de FastAPI se vuelve un tool MCP, sin reescribir la lógica. También existe `FastMCP.from_openapi(spec_dict, client=...)` a partir de una spec, sin la app en memoria.

5. MCP Toolbox for Databases — SQL sin código

- Proyecto separado de Google (`googleapis/genai-toolbox`), binario en Go.
- Convierte queries SQL declaradas en YAML en un servidor MCP — **sin escribir código**.
- **Importante:** del lado ADK **no se consume con** `McpToolset`. Se consume con `toolbox_core.ToolboxSyncClient`, que devuelve *callables* normales que ADK trata como cualquier función Python.

```yaml
tools:
  get-ticket-by-id:
    kind: postgres-sql
    source: postgresql
    parameters:
      - name: ticket_id
        type: string
    statement: SELECT * FROM tickets WHERE ticket_id = $1;   # siempre bound param
```

MCP Toolbox — cliente Python

```
from toolbox_core import ToolboxSyncClient

try:
    toolbox = ToolboxSyncClient("http://127.0.0.1:5000")
    toolbox_tools = toolbox.load_toolset("tickets_toolset")
except Exception:
    toolbox_tools = []   # el agente igual carga sin el Toolbox levantado

root_agent = Agent(
    model="gemini-2.5-flash",
    tools=[*toolbox_tools],   # se mezclan como funciones normales
)
```

`bound_params` permite fijar un parámetro (ej. `tenant_id`) sin que el LLM lo vea ni lo decida.

FastMCP vs. MCP Toolbox

	FastMCP	MCP Toolbox
Quién escribe la lógica	vos, en Python	nadie — SQL en YAML
Para qué sirve	lógica arbitraria	específicamente, bases de datos
Validación	Pydantic (type hints)	el binario, por tipos declarados en YAML
Lo mantiene	vos (o la comunidad de FastMCP)	Google, versionado, multi-DB

Gotcha real: mismatch de versión de protocolo entre binario viejo del Toolbox (v0.6.0) y cliente actual → solucionado subiendo a v1.11.0.

6. Dos ejemplos reales — panorama

	software-bug-assistant	travel-concierge
Servidor MCP	MCP Toolbox (HTTP, `:5000`)	`@openbnb/mcp-server-airbnb` (STDIO, `npx`)
Tools MCP	`get-ticket-by-id`, `create-new-ticket`, …	`airbnb_search`, `airbnb_listing_details`
Otros tools	Google Search, StackExchange (LangChain), GitHub MCP remoto	sub-agentes mockeados (`hotel_search_agent`)

Los dos son **solo clientes MCP** — nunca actúan como servidor.

El punto pedagógico central

Para el LLM y el código del agente, **no hay ninguna diferencia** entre:

- un tool nativo (`get_current_date`)
- un tool de terceros (`LangchainTool`)
- un tool MCP (`McpToolset`)
- un tool de agente (`AgentTool`)

Todos son entradas de la misma lista `tools=[...]`. MCP no cambia el contrato con el modelo — cambia **de dónde viene la implementación** y **cómo se transporta** la llamada.

Cierre

- `FunctionTool` → función Python nativa (explícito solo hace falta para `require_confirmation` o `LongRunningFunctionTool`)
- `AgentTool` → agente completo como tool (llamada, no transferencia de turno)
- `McpToolset` → cliente MCP genérico (STDIO / SSE / HTTP streaming)
- `FastMCP` → framework para escribir el servidor MCP propio
- `MCP Toolbox` → SQL declarativo → servidor MCP, sin código, consumido vía `toolbox_core`

Todo verificado contra `google-adk` 2.2.0 y `fastmcp` 3.4.2 instalados en los venvs del módulo.